

Android 驱动调试指南

公司： 深圳贝特莱电子科技股份有限公司

版本： V1.1

时间： 2021/06/11

前言

为了更好服务客户，加快客户对于我司触摸 IC 驱动调试进度，节省调试时间，助力驱动开发人员快速完成我司触控驱动调试或者移植工作，该文档将从驱动目录结构、驱动移植步骤来介绍我司触控系列 android 驱动。

驱动文件结构

```
xucs@touchcompileservers:~/project/betterlife_ts$ tree
├── bl_chip_common.h
├── bl_chip_custom.h
├── bl_fw.h
├── bl_ts.c
├── bl_ts.h
├── btl_touch.dtsi
├── factory_test
│   ├── bl_test.c
│   ├── bl_test.h
│   ├── bl_test_ini.c
│   ├── bl_test_ini.h
│   ├── bl_test_msc.c
│   ├── bl_test_sc.c
│   ├── btl_msc.ini
│   ├── btl_sc.ini
│   └── Makefile
├── fw_update.c
└── Makefile

1 directory, 17 files
```

bl_chip_common.h:该文件定义了我司触摸 IC 相关的信息，以及依赖于不同应用模块所需的宏定义。

bl_chip_custom.h:该文件集合了驱动基本配置以及各应用子模块宏定义。

bl_fw.h:该文件存放触摸固件数据

bl_ts.c:该文件是驱动主体文件，实现了我司触摸 IC 相关子应用模块流程以及抽象了必需接口函数。

bl_ts.h:该文件包含了平台头文件、相关数据类型、相关引用接口函数。

btl_touch.dtsi:包含驱动文件使用的 dts 配置参考文件。

factory_test:整机厂测模块目录。

..... bl_test.c:厂测主体文件。

..... bl_test.h:厂测头文件

..... bl_test_ini.c:实现了厂测模块解析 ini 配置文件的接口函数。

。。。。。 bl_test_ini.h:解析 ini 配置文件所需的头文件。

。。。。。 bl_test_mcsc.c:自互一体触摸 IC 的厂测流程。

。。。。。 bl_test_sc.c:自容触摸 IC 的厂测流程。

。。。。。 btl_mcsc.ini:自互一体厂测所需配置文件。

。。。。。 btl_sc.ini 自容厂测所需配置文件。

。。。。。 Makefile:厂测模块的编译规则文件。

fw_update.c:触摸 IC 升级流程实现文件。

Makefile:驱动主编译规则文件。

驱动移植步骤

一、 触摸 IC 基本配置

根据实际项目需求以及硬件设计，配置 bl_chip_custom.h 文件。

bl_chip_custom.h 文件宏定义内容如下：

```
xucs@touchcompileservers:~/project/betterlife_ts$ cat bl_chip_custom.h
#ifndef BL_CHIP_CUSTOM_H
#define BL_CHIP_CUSTOM_H
#define TS_CHIP BL7XX1
#define BTL_CHIP_NAME "BL646"
#define BTL_THREADED_IRQ
#define CTP_SLAVE_ADDR 0x2c
// #define BTL_POWER_CONTROL_SUPPORT
#define BTL_CHECK_CHIPID
#define RESET_PIN_WAKEUP
// #define INT_PIN_WAKEUP
#define BTL_UPDATE_FIRMWARE_ENABLE
#define BTL_DEBUG_SUPPORT
#define BTL_CONFIG_OF
// #define BTL_CTP_PRESSURE
// #define BTL_CTP_SUPPORT_TYPEB_PROTOCOL
// #define BTL_VIRTUAL_KEY_SUPPORT
// #define BTL_SYSFS_VIRTUAL_KEY_SUPPORT
// #define BTL_GESTURE_SUPPORT
// #define BTL_PROXIMITY_SUPPORT
#define BTL_ESD_PROTECT_SUPPORT
#define BTL_CHARGE_PROTECT_SUPPORT
// #define BTL_FACTORY_TEST_EN
#define BTL_SUSPEND_MODE
#define BTL_DEBUGFS_SUPPORT
#define BTL_APK_SUPPORT
#endif
```

- 1、 TS_CHIP:根据项目所使用的具体触摸 IC 型号配置此宏，具体选择哪个宏请咨询本司工程人员。本驱动目前支持的系列型号宏定义在 bl_chip_common.h 中定义：

```
xucs@touchcompileservers:~/project/betterlife_ts$ cat bl_chip_common.h | more
#ifndef BL_CHIP_COMMON_H
#define BL_CHIP_COMMON_H
#include "bl_chip_custom.h"

#define BL8XXX_60 0x01//b18105,b18265,b18335,b18495,b18810,b18818h,b18825,b18858c,b18868c
#define BL8XXX_61 0x02//b18281,b18331,b18858s,b18868s
#define BL8XXX_63 0x03//b18668,b18678,b18818,b18858h,b18868h,b18878
#define BL6XX0 0x04//b16090,b16130,b16280,b16360
#define BL6XX1 0x05//b16391
#define BL6XX3 0x06//b16131z,b16133u,b1m13
#define BL6XX6 0x07//b17450
#define BL7XX0 0x08//b17450
#define BL7XX1 0x09
#define BL7XX3 0x0a
```

- 2、 BTL_CHIP_NAME:该宏定义具体使用的触摸 IC 型号，可以查看 IC 丝印或者咨询我司工程人员。
- 3、 BTL_THREADED_IRQ: 中断机制为中断线程方式
对于中断处理有以下 3 种方式，如下：

- (1)、BTL_THREADED_IRQ: 中断线程方式(建议使用)
- (2)、BTL_WAIT_QUEUE: waiter 队列机制
- (3)、BTL_WORK_QUEUE: 工作队列机制
- 4、CTP_SLAVE_ADDR:配置设备 I2C 地址(7 位地址),不建议修改。
- 5、BTL_POWER_CONTROL_SUPPORT: 该宏根据硬件实际设计进行配置, 若 TP 供电可控请开启此宏, 并且对 bl_chip_common.h 文件中 BTL_POWER_CONTROL_SUPPORT 包含的宏进行配置, 宏定义如下:

```
/******Betterlife ic update******/  
#ifdef BTL_POWER_CONTROL_SUPPORT  
//#define BTL_VCC_SUPPORT  
#if defined(BTL_VCC_SUPPORT)  
//#define BTL_VCC_LDO_SUPPORT  
//#define BTL_CUSTOM_VCC_LDO_SUPPORT  
#endif  
//#define BTL_IOVCC_SUPPORT  
#if defined(BTL_IOVCC_SUPPORT)  
//#define BTL_IOVCC_LDO_SUPPORT  
//#define BTL_CUSTOM_IOVCC_LDO_SUPPORT  
#endif  
#endif
```

- (1)、vcc 可控, 请打开 BTL_VCC_SUPPORT,否则, 关闭
 - 1、BTL_VCC_LDO_SUPPORT: 标识 vcc 使用平台 LDO 控制
 - 2、BTL_CUSTOM_VCC_LDO_SUPPORT: 标识 vcc 使用其它电源供电
- (2)、iovcc 可控, 请打开 BTL_IOVCC_SUPPORT,否则, 关闭
 - 1、BTL_IOVCC_LDO_SUPPORT: 标识 iovcc 使用平台 LDO 控制
 - 2、BTL_CUSTOM_IOVCC_LDO_SUPPORT: 标识 iovcc 使用其它电源供电

注意：

- 1、 请根据具体硬件设计配置以上电源相关宏开关
 - 2、 请根据具体硬件设计配置 `dtb`
 - 3、 请根据具体硬件设计以及 `dtb` 配置在 `bl_ts.c` 文件中对
`btl_parse_dt` 、 `btl_request_platform_resource` 、
`btl_vcc_on`、`btl_vcc_off`、`btl_iovcc_on`、`btl_iovcc_off`
接口进行对应实现。
- 6、 `BTL_CHECK_CHIPID`：此宏开启将在驱动初始化中读取 ic 的设备 ID。具体 ID 值依赖于所配置 `TS_CHIP` 宏，具体值可以查看 `bl_chip_common.h` 中对应宏包含的 `BTL_FLASH_ID` 宏定义。

注意：

该功能可以确认通讯是否正常，硬件是否基本正常的依据

- 7、 `RESET_PIN_WAKEUP`:支持 `reset` 引脚唤醒。
- 8、 `INT_PIN_WAKEUP`:支持 `int` 脚唤醒。(不建议使用)
- 9、 `BTL_UPDATE_FIRMWARE_ENABLE`：开启此宏，将在开机执行升级固件流程。

注意：

- 1、 升级功能包含 2 种升级方式，在 `bl_chip_common.h` 中进行单独配置，具体如截图：

```

/*****Betterlife ic update*****/
#ifndef BTL_UPDATE_FIRMWARE_ENABLE
#define BTL_FWVER_MAIN_OFFSET (0x2a)
#define BTL_FWVER_ARGU_OFFSET (0x2b)
#define BTL_PROJECT_ID_OFFSET (0x2c)
#define BTL_PROJECT_INFO_OFFSET (0x0f)
#define BTL_COB_ID_OFFSET (0x34)
#define BTL_COB_ID_LEN (12)
#define BTL_ARGUMENT_FLASH_SIZE (1024)
#define PRJ_INFO_LEN (0x08)
#define FLASH_PAGE_SIZE (512)
//Update firmware through driver probe procedure with h and c file
//#define BTL_AUTO_UPDATE_FIRMWARE
#endif
//Update firmware through adb
#define BTL_UPDATE_FIRMWARE_WITH_BIN
#ifdef BTL_UPDATE_FIRMWARE_WITH_BIN
#define BTL_FIRMWARE_BIN_PATH "/sdcard/BTL.bin"
#endif
#endif

```

(1)、BTL_AUTO_UPDATE_FIRMWARE:

可以通过 `bl_fw.h` 文件中合入的固件数据进行升级，即通过数组方式升级,默认关闭。

(2)、BTL_UPDATE_FIRMWARE_WITH_BIN:

前提版本是可以 `root` 以及 `remount` 版本，且整个过程不能休眠，然后将 `bin` 格式的固件 `push` 到手机 `sdcard` 目录下，将名称改为 `BTL.bin`，然后通过 `adb` 进行升级，此操作过程如下：

```
adb push *.bin /sdcard/BTL.bin
```

```
cd sys/betterlife_ts
```

```
echo 1 > fwUpdate
```

- 10、BTL_DEBUG_SUPPORT: 是否开启驱动日志打印功能
- 11、BTL_CONFIG_OF: 是否需要 `dts` 支持
- 12、BTL_CTP_PRESSURE: 目前我司触摸 IC 暂不支持。
- 13、BTL_CTP_SUPPORT_TYPEB_PROTOCOL: 是否支持 B 协议数据上报方式，默认使用关闭，使用 A 协议。

- 14、 BTL_VIRTUAL_KEY_SUPPORT: 是否支持虚拟按键
- 15、 BTL_SYSFS_VIRTUAL_KEY_SUPPORT: 虚拟按键的另一种实现方式, 将按键配置信息写入 board_properties 进而完成按键上报功能
- 16、 BTL_GESTURE_SUPPORT: 是否支持手势功能
- 17、 BTL_PROXIMITY_SUPPORT: 是否支持模拟接近感应功能
- 18、 BTL_ESD_PROTECT_SUPPORT: 是否需要 esd 检测机制
- 19、 BTL_CHARGE_PROTECT_SUPPORT: 是否需要充电保护机制
- 20、 BTL_FACTORY_TEST_EN: 是否需要整机厂测功能
- 21、 BTL_SUSPEND_MODE: 休眠模式选择

在此宏开启的前提下, 需要根据平台实际应用场景在 bl_chip_common.h 中进行对应的休眠唤醒流程宏控开关:

```
/******Betterlife driver suspend mode select *****/  
#if defined(BTL_SUSPEND_MODE)  
//#define BTL_SYSFS_SUSPEND_SUPPORT  
#define BTL_EARLYSUSPEND_SUPPORT  
//#define BTL_FB_SUPPORT  
//#define BTL_PM_SUPPORT  
//#define BTL_ADF_SUPPORT  
#endif
```

BTL_SYSFS_SUSPEND_SUPPORT:通过系统对节点操作实现休眠唤醒, 如果选择此方式, 还需要对 bl_ts.c 文件中对应的节点名称进行确认。

BTL_EARLYSUSPEND_SUPPORT: earlysuspend 机制休眠唤醒

BTL_FB_SUPPORT: FB 机制休眠唤醒

BTL_PM_SUPPORT: 电源系统机制休眠唤醒

BTL_ADF_SUPPORT: adf 机制休眠唤醒

注意：

如果平台是通过其它方式进行休眠唤醒，请自行实现新方式休眠唤醒流程，最终需要调用我司休眠唤醒接口，分别是：

btl_ts_suspend： 休眠接口

btl_exit_sleep： 唤醒接口

22、 BTL_DEBUGFS_SUPPORT：用于调试用的 sysfs 系统，具体路径如下：

```
l7156: sys # ls -la
total 4
dr-xr-xr-x 16 root root 0 2021-12-24 15:29 .
drwxr-xr-x 21 root root 4096 2021-12-20 14:38 ..
drwxr-xr-x 2 root root 0 2021-12-24 15:29 betterlife ts
drwxr-xr-x 2 root root 0 2021-12-24 15:29 block
drwxr-xr-x 2 root root 0 2021-12-24 15:29 bootinfo
drwxr-xr-x 21 root root 0 2021-12-24 15:29 bus
drwxr-xr-x 90 root root 0 2021-12-24 15:29 class
drwxr-xr-x 4 root root 0 2021-12-24 15:29 dev
drwxr-xr-x 10 root root 0 2021-12-24 15:29 devices
drwxr-xr-x 3 root root 0 2021-12-24 15:29 firmware
drwxr-xr-x 8 root root 0 2021-12-24 15:29 fs
drwxr-xr-x 12 root root 0 2021-12-24 15:29 kernel
drwxr-xr-x 142 root root 0 2021-12-24 15:29 module
drwxr-xr-x 2 root root 0 2021-12-24 15:29 mtk_rgu
drwxr-xr-x 5 root root 0 2021-12-24 15:29 power
lrwxrwxrwx 1 root root 0 2021-12-24 15:29 touchscreen -> betterlife_ts
drwxr-xr-x 2 root root 0 2021-12-24 15:29 vcodec
l7156:/sys #
```

节点列表如下：

```
sp9832e_1h10_go:/sys/betterlife_ts # ls
chargeEn driver_version fwUpdate int_op logEn pcDebugOp reg special_reg ts_suspend
chipID esdEn fwVersion irqEn pcConStatus pcTransLen rst_op systest
sp9832e_1h10_go:/sys/betterlife_ts #
```

btl_test: 厂测测试节点，在常亮屏前提下，首先将我司发布的节点卡控配置文件 push 到设备 sdcard 目录下面，格式是 ini 格式，例如 btl_mcsc.ini 文件，命令序列如下：

adb push btl_mcsc.ini /sdcard/

echo "btl_mcsc.ini" > btl_test

等待结束后，会在 `sdcard` 目录下生成结果文件，一般以 `FAIL` 或者 `PASS` 开头的 `txt` 格式文件，见下图：

```
Android DCIM MOVIES Notifications Podcasts BTL_mcsc.ini
17156:/sys/betterlife_ts # echo "btl_mcsc.ini" > btl_test
17156:/sys/betterlife_ts # ls /sdcard/
Alarms Download Notifications btl_mcsc.ini
Android FAIL_1640336747_testresult.txt Pictures
BTL.bin MOVIES Podcasts
DCIM Music Ringtones
17156:/sys/betterlife_ts # |
```

chipID:读取设备 ID，操作如下图：

```
17156:/sys/betterlife_ts # cat chipID
ChipID = 41
17156:/sys/betterlife_ts # _
```

Driver_version:查询驱动发布版本号

irqEn: 写 0 禁止中断，写 1 使能中断

esdEn: 写 0 关闭 esd 功能模块，写 1 使能 esd 功能模块

chargeEn: 写 0 关闭充电检测功能模块，写 1 使能充电检测模块

pcConStatus: 用于 PC 辅助调试工具

pcTransLen: 用于 PC 辅助调试工具

pcDebugOp: 用于 PC 辅助调试工具

注意：

`pcConStatus` ， `pcTransLen` ， `pcDebugOp` 依赖于 `BTL_PC_DEBUG_SUPPORT`，而 `BTL_PC_DEBUG_SUPPORT` 依赖于 `BTL_DEBUGFS_SUPPORT`

fwUpdate: 前面已有描述，这不再描述

fwVersion: 读取版本号，如下：

```
l7156:/sys/betterlife_ts # cat fwVersion
fwVer:1
argVer:1
l7156:/sys/betterlife_ts # _
```

int_op: 操作主控中断脚

0: 输出低电平

1: 输出高电平

-1: 输入

logEn:关闭和开启触摸驱动 log

0:关闭

1: 开启

log_level: 设置厂测模块日志等级，越大日志等级越高

rawdata_status: 查询厂测结果

reg: 读写寄存器操作，具体协议见下图:

A	B	C	D	E	F
读操作	1	IIC总线发送的数据字节数	data0...dataN	IIC总线要接收的数据字节数	0(十六进制字节显示) 1(无符号十进制2字节合并) 2(无符号十进制2字节合并)
1、写操作, 对a5寄存器写03根据上述协议命令串如下: echo "0,2,a5,03" > reg 2、读操作, 读取版本号根据上述协议命令串如下: echo "1,1,b6,3,0" > reg					

rst_op:主控复位引脚操作

0: 复位脚输出低电平

1: 复位引脚输出高电平

special_reg: 但读取 diff、rawdata 等数据时需要此节点，替

协议见下图:

SPECIAL_REG操作协议			
(读写操作)OP	寄存器地址	LEN(IIC总线上要读取的字节数)	数据处理模式
读操作	要读取的寄存器地址	IIC总线要接收的数据字节数	0(十六进制字节显示) 1(无符号十进制2字节合并) 2(有符号十进制2字节合并)

读操作, 对1f寄存器进行读操作:
 注意:
 数据处理模式为0:
 读取的数据回显个数是传入的长度LEN
 数据处理模式为1或者2时:
 读取的数据回显个数是按照tx和rx来自的计算个数, 主要针对rawdata\diff等占2个字节的数据例如:
 echo "1f,tx*rx*2,1" > reg

vcc_op: 如果 TP 的 vcc 供电可控, 则通过此节点可以关闭

vcc 和开启 vcc

0: 关闭 vcc

1: 打开 vcc

23、 BTL_APK_SUPPORT: 支持 apk 读取数据进行调试, 开启此宏将在 proc 目录下生成 Betterlife_ts 节点, 如图:

```

17156:/proc # ls
1      204    271    37     560    83      loadavg
10     205    272    370    561    84      locks
100    206    273    372    562    842     m4u
101    207    27312  373    563    845     mali
102    208    27383  375    565    847     mcdi
103    209    274    376    566    85      meminfo
105    210    27484  378    568    850     mfb
107    211    275    379    57     858     misc
1077   213    276    38     572    86      mobile_tm
1078   214    27691  380    573    864     modules
108    2146   277    381    577    865     mounts
109    215    278    382    579    868     mrdump_rst
11     216    27848  383    58     87      msdc_debug
110    217    279    384    580    88      msdc_help
1109   2178   28     386    581    89      mt-auxadc
111    218    280    39     585    9       mt_power_gs
112    2185   281    4      59     90      mtd
113    219    284    40     590    91      mtk-perf
12     21902  28521  409    591    92      mtk_battery_cmd
1250   21999  287    42     592    93      mtk_cmdq_debug
1259   22     288    424    593    94      mtk_jpeg
13     220    289    425    594    942     mtkcooler
1320   221    29     426    6      95      mtktz
1339   222    290    428    60     96      mtprintk
135    223    291    429    600    98      mtprof
136    224    29163  43     601    99      net
1369   225    292    430    602    99      owe
1398   226    293    431    603    99      vnet
  
```

注意:

1、 软件版本必须为 root 版本

2、 软件版本必须关闭 selinux 权限

3、 软件版本必须可以 remount

24、 BTL_TOUCHPAD_SUPPORT:开启此宏支持触摸板项目数据报点协议上报

25、 BTL_PC_DEBUG_SUPPORT: 开启此宏驱动将提供 PC 调试工具所需的接口

二、 esd 简要说明

开启 esd 后，主控将 3 秒钟一次读取 f9 寄存器，此寄存器每次读取的值会不一样，如一样则发生异常需要 reset 恢复或者上下电恢复

三、 充电保护机制简要说明：

开启充电保护后，触摸驱动需要获取主控充电状态，因此需要在 bl_ts.c 中实现 btl_get_charge_status 接口。固件会 3 秒一次获取充电状态，根据是否充电对 0x8b 寄存器进行操作

四、 手势功能简要说明：

但需要手势时，在系统休眠时主控会对 0xd0 写 1 通知固件进入手势模式，如有中断上报，主控读取 0x01 寄存器获取手势码。退出睡眠时，主控向 0xd0 写 0 通知固件退出手势模式进入正常报点模式，主控收到中断读取 0x01 寄存器获取坐标相关信息

五、 休眠模块

主控向 0xa5 寄存器写 0x03 可以使触摸 IC 进入 sleep 状态

六、 唤醒模块

主控通过 reset 或者 int 引脚唤醒触摸 IC，推荐使用 reset 唤醒,int 唤醒方式对时序条件严苛且增加软件流程复杂度。

七、 报点协议格式说明

1、 触摸屏协议格式表

数据地址	B7	B6	B5	B4	B3	B2	B1	B0
0x01	Gesture Code (手势码)							
0x02	Point Number (手指个数)							
6*n + 0x03	EventFlag (手指 触摸状态) 00: Down 01: Up 10: Contact or Move 11: Reserved		XH (X 坐标的高 6 位数据)					
6*n + 0x04	XL (X 坐标的低 8 位数据)							
6*n + 0x05	TouchID (手指的 Tracking ID)				YH (Y 坐标的高 4 位数据)			
6*n + 0x06	YL (Y 坐标的低 8 位数据)							
6*n + 0x07	Reserved							
6*n + 0x08	Reserved							

主控通过 IIC 总线读取 01 寄存器，读取到的数据根据上表进行触摸信息解析并上报。

2、 触摸板协议格式说明

(1)、报点协议说明

地址	名称	Bit7	Bit6	Bit5~3	Bit2	Bit1	Bit0	缺省值
0x01	手势码	详见 5.3 手势码表						0
0x02		水平 Flag	垂直 Flag		中键	右键	左键	0
0x03	Delta X	-127~127						0
0x04	Delta Y	-127~127						0
0x05	Delta Z	-8~7						0

备注：

0x02 寄存器的 Bit7 为 1 时，滚轮为水平方向移动，读取 Delta Z 值；0x02 寄存器的 Bit6 为 1 时，滚轮为垂直方向移动，读取 Delta Z 值。

(2)、手势码表说明

放大	0x01
缩小	0x02
单指快速点击	0x03
双指快速点击	0x04
三指快速点击	0x05
四指快速点击	0x0A
一个手指从左边缘向右滑动	0x06
一个手指从右边缘向左滑动	0x07
一个手指从上边缘向下滑动	0x08
一个手指从下边缘向上滑动	0x09
三个手指同时向右划	0x10
三个手指同时向左划	0x11
三个手指同时向下划	0x12
三个手指同时向上划	0x13

主控通过 IIC 总线读取 01 寄存器，读取到的数据根据上表进行触摸信息解析并上报。

八、PC 辅助工具所需驱动解口说明

1、 pcConStatus:

向该节点写 0 将关闭驱动中断、esd、充电检测模块，写 1 将重新开启驱动中断、esd、充电检测模块，每次执行完毕后，可以通过查询获取写入指令是否执行成功，操作命令流程如截图

```
sp9832e_1h10_go:/sys/betterlife_ts # echo 1 > pcConStatus
sp9832e_1h10_go:/sys/betterlife_ts # cat pcConStatus
OK
sp9832e_1h10_go:/sys/betterlife_ts # echo 0 > pcConStatus
sp9832e_1h10_go:/sys/betterlife_ts # cat pcConStatus
OK
sp9832e_1h10_go:/sys/betterlife_ts # |
```

2、 pcTransLen:

该节点只存在读操作，返回平台单次 i2c 发送或者接受的最大字节数，返回为 16 进制格式字串，示意如截图：

```
sp9832e_1h10_go:/sys/betterlife_ts # cat pcTransLen
[80]OK
sp9832e_1h10_go:/sys/betterlife_ts # |
```

3、 pcDebugOp:

此节点主要用于 PC 辅助工具与驱动之间的具体交互业务，具体协议如下：

操作码	可选字段 0	可选字段 1	可选字段 2	……	可选字段 N
0					
1					
reserved	/	/	/	/	/

不同操作码对应具体协议

a) 操作码 = 0，表示将执行 I2C 写操作任务，协议如下

操作码	可选字段 0	可选字段 1
0	写入总线字节长度	写入具体字节数据

例如：向 8B 寄存器写 1，示例截图如下

```
sp9832e_1h10_go:/sys/betterlife_ts # echo "0,2,8b,1" > pcDebugOp
sp9832e_1h10_go:/sys/betterlife_ts # cat pcDebugOp
OK
```

写入成功，查询返回 OK,否则 NG

b) 操作码 1，表示将执行读取 I2C 读操作任务，协议如下

操作码	可选字段 0	可选字段 1	可选字段 2
1	写入总线字节长度	写入具体字节数据	要读取的字节数据长度

例如，读取 B6 寄存器 3 个字节，示例截图如下：

```
sp9832e_1h10_go:/sys/betterlife_ts # echo "1,1,b6,3" > pcDebugOp  
sp9832e_1h10_go:/sys/betterlife_ts # cat pcDebugOp  
[4,7,0]OK  
sp9832e_1h10_go:/sys/betterlife_ts # |
```

写入成功，查询返回[读取到的数据]OK,否则 NG